Q. Write short note on *LEX*.

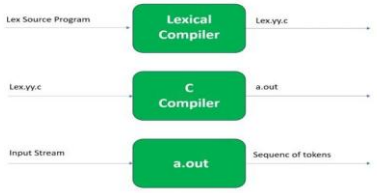
Lex is a tool or a computer program that generates Lexical Analyzers (converts the stream of characters into tokens). The Lex tool itself is a compiler. The Lex compiler takes the input and transforms that input into input patterns. It is commonly used with YACC(Yet Another Compiler Compiler). It was written by Mike Lesk and Eric Schmidt.

Function of Lex: 1. In the first step the source code which is in the Lex language having the file name 'File.l' gives as input to the Lex Compiler commonly known as Lex to get the output as lex.yy.c. 2. After that, the output lex.yy.c will be used as input to the C compiler which gives the output in the form of an 'a.out' file, and finally, the output file a.out will take the stream of character and generates tokens as output.

lex.yy.c: It is a C program.

File.l: It is a Lex source program

a.out: It is a Lexical analyzer



Lex File Format: A Lex program consists of three parts and is separated by % delimiters:-

Declarations

%%

Translation rules

%%

Auxiliary procedures

Declarations: The declarations include declarations of variables.

Transition rules: These rules consist of Pattern and Action.

Auxiliary procedures: The Auxilary section holds auxiliary functions used in the actions.

For example:

declaration

number[0-9]

%%

translation

if {return (IF);}

%%

auxiliary function

int numberSum()

Q.Describe the *structure of symbol table*

A symbol table is a data structure used by a compiler to store information about identifiers such as variables, functions, constants, and parameters in a program.

1.Stores information such as the name, type, scope, and memory location of each identifier. 2.The symbol table is built during the early phases of compilation and is used throughout all compiler phases for error detection, scope resolution, optimization, and code generation.

Role of Symbol Table in Compiler Phases

It is used by various phases of the compiler as follows:-

Lexical Analysis: 1.Inserts identifiers into the symbol table.

2.Records token attributes like identifier names.

Syntax Analysis: 1.Updates attribute information such as scope and structure.

2.Maintains hierarchical scope information.

Semantic Analysis: 1.Performs type checking. 2.Verifies declarations before use.

3.Validates function parameters and return types.

Intermediate Code Generation: 1.Uses type and size information.

2.Adds temporary variables into the symbol table.

Code Optimization: 1.Uses scope and type information.

2.Assists in memory and register optimization.

Target Code Generation: 1.Uses memory location and offset information.

2.Generates correct machine instructions.

Example of Using Symbol Table:

: distance (variable in main), pi (constant)

radius (parameter in calculateArea), calculateArea (function)

A symbol table of this program would look like the following.

Name	Type	Scope	Category	Additional Info
pi	float	Global	Constant	Read-only
calculateArea	float	Global	Function	Returns float
radius	float	Local (Function)	Parameter	Scope: calculateArea
distance	float	Local (main)	Variable	Stack allocated
area	float	Local (main)	Variable	Stack allocated

Q. Explain *compiler vs interpreter vs translator*.-

 A *COMPILER* is a program that translates the entire source code into machine code at once.

 An *INTERPRETER* translates and executes the program line-by-line.

 A *TRANSLATOR* is a general term for any program that converts code from one language to another (includes compiler and interpreter).

Feature	Compiler	Interpreter	Translator
Translation	Whole program at once	Line-by-line	Converts code
Output	Machine code generated	No object code	May or may not Generated code
Speed	Faster execution	Slower execution	Depends on type
Error Detection	After compilation	During execution	Depends on tool
Example	C, C++ compiler	Python, JS	Compiler/Interpreter both

3. Explain input buffering technique.- I/O buffering is a technique used in computer systems to improve the efficiency of input and output (I/O) operations. It involves the temporary storage of data in a buffer, which is a reserved area of memory, to reduce the number of I/O operations and manage the flow of data between fast and slow devices or processes

Uses of I/O Buffering: 1. Buffering is done to deal effectively with a speed mismatch between the producer and consumer of the data stream.

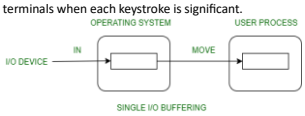
2.When the first buffer is filled, then it is requested to transfer the data to disk.

3., thus minimizing the time requirements between them.

Types of I/O Buffering Techniques: 1. Single Buffer- Using one buffer to store data temporarily. A buffer is provided by the operating system to the system portion of the main memory.

Block Oriented Device: 1. The system buffer takes the input. 2.OS can swap the processes. 3.OS can record the data of the system buffer to user processes.

Stream Oriented Device:1. Line- at a time operation is used for scroll-made terminals. The user inputs one line at a time, with a carriage return signaling at the end of a line. 2.Byte-at-a-time operation is used on forms mode, terminals when each keystroke is significant.



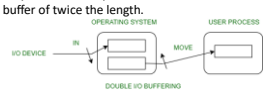
2. Double Buffer: In this technique the operating system Uses two buffers to allow continuous data transfer between two process or two devices.

Block Oriented: 1. There are two buffers in the system.

3.Double buffering is also known as buffer swapping.

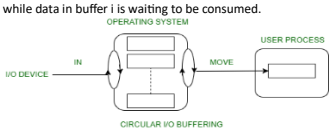
Stream Oriented:1. Line- at a time I/O, the user process need not be suspended for input or output, unless process runs ahead of the double buffer.

2.Byte- at a time operations, double buffer offers no advantage over a single buffer of twice the length.



3. Circular Buffer: In this technique the operating system Uses a circular buffer to manage continuous data streams efficiently.

1.When more than two buffers are used, the collection of buffers is itself referred to as a circular buffer. 2.The producer can only fill up to buffer i-1 while data in buffer i is waiting to be consumed.



Q. Compare top-down and bottom-up parsing strategies.- Top-Down Parsing- Top-down parsing on the other hand is a parsing technique whereby the parse tree is grown from the top down from the start symbol as the root moving down to the terminal symbols at the bottom. The parser attempts to translate the input string applying all the productions of the given grammar with the help of a start symbol and making iterations for the non-terminal symbols

Bottom-Up Parsing- The bottom up parsing is a parsing strategy in which the parse tree is constructed in a bottom of the tree and gradually travel upwards to the root. A parser attempts to analyze the input string and put it into the from of the start symbol by applying reverse production rules.

Top-Down Parsing- 1.It is a parsing strategy that first looks at the highest level of the parse tree and works down the parse tree by using the rules of grammar.

2. Top-down parsing attempts to find left most derivations for an input string.

3. in this parsing technique we start parsing from the top (start symbol of parse tree) to down (the leaf node of parse tree) in a top-down manner.

4. This parsing technique uses Left Most Derivation.

5. The main leftmost decision is to select what production rule to use in order to construct the string. 6. Example: Recursive Descent parser.

Bottom-Up Parsing- 1.It is a parsing strategy that first looks at the lowest level of the parse tree and works up the parse tree by using the rules of grammar.

2.Bottom-up parsing can be defined as an attempt to reduce the input string to the start symbol of a grammar.

3.In this parsing technique we start parsing from the bottom (leaf node of the parse tree) to up (the start symbol of the parse tree) in a bottom-up manner.

4.This parsing technique uses Right Most Derivation.

5.The main decision is to select when to use a production rule to reduce the string to get the starting symbol. 7. Example: Shift Reduce parser.

Differences between Synthesized and Inherited Attributes

Synthesized Attributes- 1. An attribute is synthesized if its value at a parse tree node is determined from the attribute values of its child nodes.

2. The production must have a non-terminal as its head.

3. A synthesized attribute at node n is defined using attribute values of the children of n. 4. Evaluated using a bottom-up traversal of the parse tree.

5. Can be associated with both terminals and non-terminals.

6. Used in **S-attributed SDT** and **L-attributed SDT**.

Inherited Attributes- 1. An attribute is inherited if its value at a parse tree node is determined from the attribute values of its parent and/or sibling nodes.

2. The production must have a non-terminal in its body.

3. An inherited attribute at node n is defined using attribute values of the parent, node itself, and siblings.

4. Evaluated using a top-down or sideways traversal of the parse tree.

5. Associated only with non-terminals. 6. Used only in **L-attributed SDT**.

Differentiate between S-attributed and L-attributed definitions.

S-ATTRIBUTED DEFINITION is a syntax-directed definition that uses only synthesized attributes.

L-ATTRIBUTED DEFINITION is a syntax-directed definition that uses both synthesized and inherited attributes, with certain restrictions.

Feature	S-Attributed	L-Attributed
Attributes Used	Only synthesized	Synthesized + inherited
Evaluation	Bottom-up parsing	Top-down parsing
Complexity	Simple	More complex
Dependency	Child → Parent	Parent → Child allowed
Parser Type	LR parser	LL parser
Flexibility	Less flexible	More flexible

What is an Abstract Syntax Tree?- An Abstract Syntax Tree, or AST, is a representation of the structure of a programming language. It is a tree-like structure that is composed of nodes, each of which represents a language element like a variable, operator, or keyword. ASTs are often used to represent code written in a specific programming language, such as Python or C++.

What is a Parse Tree?-A Parse Tree, or PT, is a tree-like structure that represents the syntactic structure of a programming language. It is composed of nodes, each of which corresponds to a language element such as a variable, operator, or keyword. Unlike ASTs, PTs are not abstracted away from the details of the language syntax. This means that a PT is a more detailed representation of the code, which can be useful for debugging.

Abstract Syntax Tree vs Parse Tree:

Factor: 1. Structure, 2.Generation 3.Usage 4.Size 5.Complexity6.Comprehension

Abstract Syntax Tree- 1. An AST is composed of symbols, such as operators, identifiers, and keywords. These symbols are organized in a tree structure.

2. An AST is generated by the compiler from the source code.

3. An AST is used to analyze the source code and optimize it for compilation.

4. An AST is smaller than a parse tree, as it contains fewer symbols.

5. An AST is simpler than a parse tree, as it is generated from fewer operations.

6. An AST is easier to comprehend than a parse tree, as it contains more information about the source code.

Parse Tree: 1. A parse tree is composed of tokens, such as terminal and non-terminal symbols. These tokens are organized in a tree structure.

2. A parse tree is generated by the parser from the source code.

3. A parse tree is used to check the syntactic correctness of the source code.

4. A parse tree is larger than an AST, as it contains more tokens.

5. A parse tree is more complex than an AST, as it is generated from more operations. 6. A parse tree is harder to comprehend than an AST, as it contains less information about the source code.

8. Explain the following: - Translators, Compilers, and Interpreters

- Bootstrapping in compiler design

1. Translators, Compilers, and Interpreters

In computer science, a **Translator** is a broad term for any program that converts source code (written in a high-level programming language) into an equivalent form that a machine can execute (machine code/binary).

Compiler: A compiler is a translator that converts the **entire** source code into a standalone machine-executable file (like a .exe on Windows) before the program is ever run. **Workflow:** Source Code \rightarrow Compiler \rightarrow Object/Machine Code \rightarrow Executable.

Key Characteristic: Once compiled, the program can be run multiple times without needing the original source code or the compiler present.

Speed: Very fast execution because the translation work is done entirely upfront. **Examples:** C, C++, Rust, Go.

Interpreter An interpreter translates and executes the source code **line-by-line** at runtime. It does not create a separate executable file.

Workflow: Source Code \rightarrow Interpreter \rightarrow Direct Execution.

Key Characteristic: The source code must be present every time the program runs, and the interpreter must be active to translate it on the fly.

Speed: Generally slower during execution, as the "translation" happens simultaneously with the processing of the code.

Examples: Python, JavaScript, Ruby, PHP.

2. Bootstrapping in Compiler Design- Bootstrapping is a fascinating concept in compiler construction: it is the process of using a compiler (or part of one) to compile itself. **The Problem: The "Chicken and Egg" Dilemma**

To write a compiler for a new language (let's call it "Language X"), you need a compiler that is already running on your machine. But if Language X is new, no such compiler exists yet. **The Solution: The Bootstrap Process**

Bootstrapping allows developers to write the compiler in the language it is meant to compile, even before the compiler exists. It typically follows these phases: **Phase 1 (The Seed):** A minimal, simplified version of the compiler for "Language X" is written in an existing, mature language (like C or Assembly).

Phase 2 (The Implementation): The developers then write the *full-featured* version of the compiler for "Language X" using "Language X" itself.

Phase 3 (Compiling the Compiler): They use the "Seed" compiler (from Phase 1) to compile the "Full-featured" source code (from Phase 2). The output is a functional, native compiler for "Language X."

Phase 4 (Self-Sustenance): The new "Full-featured" compiler can now compile its own source code, making the initial "Seed" compiler obsolete.

Q. What do you mean by context-free grammar? Give distinctions between regular and context-free grammars and mention the limitations of context-free grammars. A **Context-Free Grammar (CFG)** is a formal system used to define the syntax of programming languages and natural languages. It is called "context-free" because the replacement of a non-terminal symbol with a string of symbols can occur regardless of the symbols that surround it—the "context" does not matter.

A CFG is defined by a 4-tuple G = (V, \Sigma, R, S),

Distinctions: Regular vs. Context-Free Grammars

The primary difference lies in the complexity of the languages they can describe, which is governed by the **Chomsky Hierarchy**.

Feature,	Regular Grammar,	Context-Free Grammar (CFG)
Complexity,	Simple (Type-3),	Moderate (Type-2)
Parsing Engine,	Finite Automata (DFA/NFA),	Pushdown Automata (PDA)
Memory,	None (cannot count),	Stack (can count/match nested structures)
Capability,	"Recognizes flat, sequential patterns.",	"Recognizes recursive, nested structures."

Regular Grammars are restricted: a non-terminal can only appear at the far left or far right of a production rule. They cannot track arbitrary levels of nesting.

CFGs allow non-terminals to appear anywhere in the string, enabling them to handle recursive structures like matching parentheses or if-else blocks.

Limitations of Context-Free Grammars: While CFGs are powerful enough to describe most programming language syntax, they have distinct limitations:

Cannot Verify "Declaration Before Use": CFGs cannot enforce constraints that require checking information outside the immediate structure. For example, a CFG can define the structure of a variable declaration, but it cannot verify if a variable was declared before it was used in an expression. This requires **Semantic Analysis** using a symbol table.

Cannot Handle Cross-Serial Dependencies: CFGs struggle with languages that require matching three or more independent sets of. Because a Pushdown Automaton only has one stack, it can match two things but it cannot "remember" the count for a third set (c^n).

Ambiguity: CFGs can be inherently ambiguous, meaning one string could have two different valid parse trees. While some can be rewritten to be unambiguous, others cannot, making them difficult for parsers to interpret consistently.

Inefficiency for Complex Contexts: As the rules for context-sensitive dependencies grow, CFGs become extremely bloated and unmaintainable. That is why compilers use a hybrid approach: CFGs for structure and Attribute Grammars or Symbol Tables for context-sensitive checks.

Explain *loop optimization*.

Loop Optimization is the process of increasing execution speed and reducing the overheads associated with loops. It plays an important role in improving cache performance and making effective use of parallel processing capabilities. Most execution time of a scientific program is spent on loops.

Loop Optimization is a machine independent optimization. Whereas Peephole optimization is a machine dependent optimization technique.

Decreasing the number of instructions in an inner loop improves the running time of a program even if the amount of code outside that loop is increased.

Loop Optimization Techniques

In the compiler, we have various loop optimization techniques, which are as follows:

1. Code Motion (Frequency Reduction)

In frequency reduction, the amount of code in the loop is decreased. A statement or expression, which can be moved outside the loop body without affecting the semantics of the program, is moved outside the loop.

Example: Before optimization:

while(i<100) { a = Sin(x)/Cos(x) + i; i++; }

After optimization:

t = Sin(x)/Cos(x); while(i<100) { a = t + i; i++; }

2. Induction Variable Elimination

If the value of any variable in any loop gets changed every time, then such a variable is known as an induction variable. With each iteration, its value either gets incremented or decremented by some constant value.

Example: Before optimization:

B1 i:= i+1 x:= 3*I y:= a[x] if y< 15, goto B2

In the above example, i and x are locked, if i is incremented by 1 then x is incremented by 3. So, i and x are induction variables.

After optimization: B1 i:= i+1 x:= x+4 y:= a[x] if y< 15, goto B2

3. Strength Reduction

Strength reduction deals with replacing expensive operations with cheaper ones like multiplication is costlier than addition, so multiplication can be replaced by addition in the loop. **Example: Before optimization:** while (x<10) {

y := 3 * x+1; a[y] := a[y]-2; x := x+2; }

After optimization: t= 3 * x+1; while (x<10) { y=t;

a[y]= a[y]-2; x=x+2; t=t+6; }

Briefly describe peephole optimization with a simple example.

Peephole optimization is a type of code Optimization performed on a small part of the code. It is performed on a very small set of instructions in a segment of code. *The small set of instructions or small part of code on which peephole optimization is performed is known as **peephole** or **window**.*

It basically works on the theory of replacement in which a part of code is replaced by shorter and faster code without a change in output. The peephole is machine-dependent optimization.

Objectives of Peephole Optimization: The objective of peephole optimization is as follows: 1.To improve performance2. To reduce memory footprint 3. To reduce code size

Peephole optimization improves the efficiency of compiled code by eliminating unnecessary instructions in small code sequences. It's a key technique in modern compiler design.

Peephole Optimization Techniques

A. Redundant load and store elimination: In this technique, redundancy is eliminated. **Initial code:** y = x + 5; i = y; z = i; w = z * 3;
Optimized code: y = x + 5; w = y * 3; //* there is no i now
//* We've removed two redundant variables i & z whose value were just being copied from one another.

B. Constant folding: The code that can be simplified by the user itself, is simplified. Here simplification to be done at runtime are replaced with simplified code to avoid additional computation. **Initial code:** x = 2 * 3;
Optimized code: x = 6;

C. Strength Reduction: The operators that consume higher execution time are replaced by the operators consuming less execution time.
Initial code: y = x * 2; **Optimized code:** y = x + x; or y = x << 1;
Initial code: y = x / 2; **Optimized code:** y = x >> 1;

D. Null sequences/ Simplify Algebraic Expressions : Useless operations are deleted. a := a + 0; a := a * 1; a := a/1; a := a - 0;

E. Combine operations: Several operations are replaced by a single equivalent operation.

F. Deadcode Elimination:- Dead code refers to portions of the program that are never executed or do not affect the program's observable behavior. Eliminating dead code helps improve the efficiency and performance of the compiled program by reducing unnecessary computations and memory usage.

Initial Code:- int Dead(void) { int a=10; int z=50; int c; c=z*5; printf(c); a=20; a=a*10; //No need of These Two Lines return 0; }
Optimized Code:- int Dead(void) { int a=10; int z=50; int c; c=z*5; printf(c); return 0; }

Describe the structure and role of symbol tables in a compiler

1. The Structure of a Symbol Table

A symbol table is designed for speed, as the compiler accesses it constantly during every phase of compilation. While the implementation varies, it generally consists of:

Identifier Name: The string literal of the variable or function (e.g., myVariable).
Data Type: The type (int, float, custom object, etc.).
Scope: Where the identifier is accessible (e.g., global, function-local).
Memory Location/Offset: Where the variable resides in memory.
Attribute Information: Additional metadata, such as function parameters, return types, or accessibility modifiers (public/private).

Organization Techniques

Linear List: Simple to implement but slow for large programs (O(n) search time).
Hash Table: most common approach, offering near O(1) average search time.
Binary Search Tree/Balanced Tree: Efficient for ordered access, providing O(\log n) performance.
Scope Stacks: To handle nested scopes (like functions inside functions), compilers often use a **stack of symbol tables**. When entering a new scope, a new table is "pushed"; when exiting, it is "popped."

2. The Role of the Symbol Table: The symbol table serves as the "source of truth" across the different phases of a compiler:

A. During Lexical & Syntax Analysis- Entry Creation: As the scanner/parser encounters a new identifier, it is inserted into the symbol table.
Declaration Checking: It checks if a variable has been declared before use. If it hasn't, the compiler reports an "Undeclared Identifier" error.

B. During Semantic Analysis- Type Checking: This is the most crucial role. The compiler looks up the identifier to verify that operations are valid for its type (e.g., ensuring you aren't trying to add a string to an int).
Scope Resolution: It ensures identifiers are accessed within their valid visibility range, enforcing rules like "no duplicate variables in the same scope."

C. During Code Generation- Memory Allocation: The compiler uses the symbol table to determine how much memory to allocate for variables and calculates their addresses relative to the base pointer or global memory space.

3. Simplified Interaction Example: If the compiler reads int counter = 0;, the sequence is: **Lexical Analyzer:** Identifies int (keyword) and counter (identifier).
Symbol Table: Checks if counter exists. If not, it adds:
Name: counter, Type: int, Scope: Current
Semantic Analyzer: Later, if the code says counter = "hello";, the compiler looks up counter, sees the type int, sees the assigned value is a string, and throws a Type Mismatch Error.

What is a basic block? Explain the transformations involved in a basic block.

A **Basic Block** is a fundamental concept in compiler design and optimization. It is a sequence of consecutive instructions in a program that has exactly one **entry point** and exactly one **exit point**.

Entry Point: The only way to enter the block is through the first instruction. There are no jumps or branches *into* the middle of the block.
Exit Point: Once the first instruction is executed, all subsequent instructions in the block are guaranteed to execute in order. The only jump or branch occurs at the very end of the block.

Identifying Basic Blocks (The Partitioning Algorithm)

To partition code into basic blocks, compilers identify **"Leaders"**:
1.The first instruction is a leader. 2.Any instruction that is the target of a conditional or unconditional jump is a leader. 3.Any instruction that immediately follows a jump or return instruction is a leader.

Transformations Involved in Basic Blocks

Once the compiler has partitioned the code into basic blocks, it performs **Local Optimizations**—transformations that improve the code's efficiency without changing its overall effect.

1. Algebraic Simplification
The compiler replaces expensive operations with cheaper, mathematically equivalent ones. **Example:** Changing x = x + 0 to a "no-op" (doing nothing) or x = x * 2 to a bitwise shift x = x << 1.

2. Constant Folding
If an expression consists entirely of constants, the compiler evaluates it at compile time rather than runtime. **Example:** x = 2 * 3 + 4 becomes x = 10.

3. Copy Propagation
If a variable is assigned the value of another variable (a = b), the compiler replaces subsequent uses of a with b to eliminate the need for the assignment.
Example: * *Before:* a = b; x = a + 5; *After:* x = b + 5;

4. Dead Code Elimination: The compiler removes instructions that have no effect on the final program output. This usually happens after other optimizations leave behind variables that are no longer used.
Example: If y is calculated but never referenced again before being reassigned, the calculation is deleted.

5. Common Subexpression Elimination (CSE)
If the same expression is calculated multiple times within a block, the compiler calculates it once and stores the result in a temporary variable to reuse it.
Example: *Before:* a = b * c; x = b * c + d;
After: temp = b * c; a = temp; x = temp + d;

What are *common subexpressions*?

Code Optimization Technique is an approach to enhance the performance of the code by either eliminating or rearranging the code lines. Code Optimization techniques are as follows:
1. Compile-time evaluation 2. Common Sub-expression elimination
3.Dead code elimination 4. Code movement 5.Strength reduction

Common Sub-expression Elimination:

The expression or sub-expression that has been appeared and computed before and appears again during the computation of the code is the common sub-expression. Elimination of that sub-expression is known as **Common sub-expression elimination.** The advantage of this elimination method is to make the *computation faster and better* by avoiding the re-computation of the expression. In addition, it utilizes memory efficiently.

Types of common sub-expression elimination: The two types of elimination methods in common sub-expression elimination are:
1. Local Common Sub-expression elimination- It is used within a single basic block. Where a basic block is a simple code sequence that has no branches.
2. Global Common Sub-expression elimination- It is used for an entire procedure of common sub-expression elimination

Explain syntax-directed translation schemes in detail.-Syntax Directed Translation is a set of productions that have semantic rules embedded inside it. The syntax-directed translation helps in the semantic analysis phase in the compiler. SDT has semantic actions along with the production in the grammar. This article is about postfix SDT and postfix translation schemes with parser stack implementation of it. Postfix SDTs are the SDTs that have semantic actions at the right end of the production. This article also includes SDT with actions inside the production, eliminating left recursion from SDT and SDTs for L-attributed definitions. **Postfix Translation Schemes:**
1.The syntax-directed translation which has its semantic actions at the end of the production is called the **postfix translation scheme.**
2.This type of translation of SDT has its corresponding semantics at the last in the RHS of the production. 3.SDTs which contain the semantic actions at the right ends of the production are called **postfix SDTs.**

Example of Postfix SDT
S → A#B{S.val = A.val * B.val}
A →B@1{A.val = B.val + 1}
B →num{B.val = num.lexval}

Parser-Stack Implementation of Postfix SDTs:
Postfix SDTs are implemented when the semantic actions are at the right end of the production and with the bottom-up parser(LR parser or shift-reduce parser) with the non-terminals having synthesized attributes.
1.The parser stack contains the record for the non-terminals in the grammar and their corresponding attributes.
2.The non-terminal symbols of the production are pushed onto the parser stack.
3.Now, the LHS non-terminal and its attributes are at the top of the stack.

Production
A → BC{A.str = B.str . C.str}
B →a {B.str = a}
C →b{C.str = b}
Initially, the parser stack:
B C Non-terminals
B.str C.str Synthesized attributes
↑
Top of Stack
After the reduction occurs A →BC then after B, C and their attributes are replaced by A and in the attribute. Now, the stack:
A Non-terminals
A.str Synthesized attributes
↑
Top of stack

SDT with action inside the production:
When the semantic actions are present anywhere on the right side of the production then it is **SDT with action inside the production.**
It is evaluated and actions are performed immediately after the left non-terminal is processed.
This type of SDT includes both S-attributed and L-attributed SDTs.
If the SDT is parsed in a bottom-up parser then, actions are performed immediately after the occurrence of a non-terminal at the top of the parser stack.
If the SDT is parsed in a top-down parser then, actions are before the expansion of the non-terminal or if the terminal checks for input.

Example of SDT with action inside the production
S → A +{print '+'} B
A → {print 'num'}B
B → num{print 'num'}

Eliminating Left Recursion from SDT:
The grammar with left recursion cannot be parsed by the top-down parser. So, left recursion should be eliminated and the grammar can be transformed by eliminating it.

Grammar with Left Recursion P → Pr | q
Grammar after eliminating left recursion P → qA, A → rA | ∈

SDT for L-attributed Definitions:

Write a short note on operator precedence parsing.

Operator Precedence Parsing is a bottom-up parsing technique used for a specific class of grammars known as **Operator Grammars**. It is particularly effective for parsing mathematical expressions where the structure is defined by the hierarchy of operators (like +, *, ^).

Core Principles: The technique relies on the **precedence** and **associativity** of operators to decide when to "shift" (read a new token) and when to "reduce" (group symbols together). Unlike general bottom-up parsers that require a full set of production rules, operator precedence parsing uses a simple table to compare the precedence of two adjacent operators.

How the Parsing Works-
The parser maintains a stack and compares the operator on top of the stack (or the most recent operator) with the incoming operator from the input string:
If the incoming operator has higher precedence: The parser performs a **Shift**, pushing the operator onto the stack.
If the operator on the stack has higher or equal precedence: The parser performs a **Reduce**, grouping the operands and operators into a "handle" and replacing them with a non-terminal.

Key Requirements for Operator Grammars
For this parser to work, the grammar must follow two strict rules:
No -productions: It cannot contain empty strings.
No two adjacent non-terminals: Two non-terminals cannot appear next to each other in any production rule (e.g., is forbidden).

Advantages and Limitations
Advantages: It is very fast, easy to implement, and requires minimal memory, making it highly efficient for simple expression parsing.
Limitations: It can only handle a narrow class of grammars. It cannot parse complex programming constructs (like nested if-else or declarations) and does not provide a complete parse tree, as it ignores the specific non-terminals used in production.
This parsing style is essentially the engine behind how a calculator or a basic interpreter evaluates an expression like 3 + 4 * 5—it recognizes that * has higher precedence than + and groups the 4 * 5 first, regardless of the order they appear in the string.

4. Loop Invariant Method
In the loop invariant method, the expression with computation is avoided inside the loop. That computation is performed outside the loop as computing the same expression each time was overhead to the system, and this reduces computation overhead and hence optimizes the code.
Example: Before optimization: for (int i=0; i<10;i++) t= i+(x/y) ... end;
After optimization: s = x/y; for (int i=0; i<10;i++) t= i+ s; ... end;

5. Loop Unrolling: Loop unrolling is a loop transformation technique that helps to optimize the execution time of a program. We basically remove or reduce iterations. Loop unrolling increases the program's speed by eliminating loop control instruction and loop test instructions.
Example: Before optimization: for (int i=0; i<5; i++) printf("Pankaj\n");
After optimization: printf("Pankaj\n"); printf("Pankaj\n"); printf("Pankaj\n"); printf("Pankaj\n"); printf("Pankaj\n");

6. Loop Jamming: Loop jamming is combining two or more loops in a single loop. It reduces the time taken to compile the many loops.
Example: Before optimization: for(int i=0; i<5; i++) a = i + 5;
for(int i=0; i<5; i++) b = i + 10;
After optimization: for(int i=0; i<5; i++) { a = i + 5; b = i + 10; }

7. Loop Fission: Loop fission improves the locality of reference, in loop fission a single loop is divided into multiple loops over the same index range, and each divided loop contains a particular part of the original loop.
Example: Before optimization: for(x=0;x<10;x++) { a[x]=... b[x]=... }
After optimization: for(x=0;x<10;x++) a[x]=... for(x=0;x<10;x++) b[x]=...

8. Loop Interchange
In loop interchange, inner loops are exchanged with outer loops. This optimization technique also improves the locality of reference.
Example:
Before optimization:

```
for(x=0;x<10;x++)
for(y=0;y<10;y++)
a[y][x]=...
```

After optimization:

```
for(y=0;y<10;y++)
for(x=0;x<10;x++)
a[y][x]=...
```